

HYBRID CONTROL-BASED ABR: TOWARDS LOW-DELAY LIVE STREAMING

Bo Wang^{1,2}, Fengyuan Ren^{1,2}, Chao Zhou³¹Beijing National Research Center for Information Science and Technology, Beijing, China²Department of Computer Science and Technology, Tsinghua University, Beijing, China³Beijing Kuaishou Technology Co., Ltd, Beijing, China

{wangbo, renfy}@csnet1.cs.tsinghua.edu.cn, zhouchaoyf@gmail.com

ABSTRACT

Video content providers are increasingly interested in interactive live streaming since user engagement increases their revenues. To provide high quality of experience (QoE), it is critical to design a low delay adaptive bitrate (ABR) algorithm, but which is lacked in existing studies. The low delay constraint poses much more challenges to achieve high bitrate and low rebuffering. For example, low delay requires the player to maintain a small playback buffer, which, however, increases the risk of rebuffering. This work designs a low delay ABR algorithm called HCA which provides good QoE by controlling the buffer occupancy at a low but non-empty level. To achieve accurate control, HCA uses the hybrid of feedback and feedforward control to regulate the buffer dynamic (buffer occupancy and its variation) based on predictions of future network condition (throughput and its variation). Trace-driven experiments show that HCA achieves zero rebuffering for 98% of all traces while ensuring high bitrate.

Index Terms— HTTP adaptive bitrate streaming, low delay, interactive live streaming

1. INTRODUCTION

Since user engagement increases revenue, video content providers like YouTube and Twitter's Periscope are increasingly interested in interactive live streaming where viewers can interact with the broadcaster when watching video [1]. To maintain interactivity, live streaming requires the time between the event is recorded and is viewed (i.e., the delay) is low. Content providers strive to provide good quality of experience (QoE) under low delay constraint to attract more users.

HTTP Adaptive Bitrate (HAS) streaming (standardized as DASH [2]) is the primary tool to optimize QoE in varying network conditions. It splits a video into chunks with each encoded into several bitrates, and uses client-side adaptive bitrate (ABR) algorithm to dynamically adjust bitrate in the unit of chunk. Commercial players such as Netflix and YouTube use HAS to improve QoE [3]. To better support interactive streaming, HAS recently added low delay option to allow chunk to be played before completely downloaded.

Despite the broad interest in the design of ABR algorithm, existing works mainly focus on VoD streaming which has no constraint of low delay. What is critically lacking is a low delay algorithm for interactive live streaming. However, selecting the right bitrate with low delay constraint is much more challenging due to (1) the conflicts between low delay and little rebuffering (low delay requires small playback buffer occupancy while continuous playback need large buffering to absorb the impact of throughput fluctuations); (2) the difficulty of delivering high-quality video while without inducing rebuffering and increasing the delay (videos of higher bitrate requires longer time to download, which might drain out the buffer thus inducing rebuffering and increasing the delay).

Contributions. To provide good experience of both viewing and interactivity, this work designs a low delay ABR algorithm (called as HCA) based on Hybrid Control, which employs a small buffer capacity (i.e., maximum buffer size) and selects the highest safe bitrate while preventing the buffer occupancy from empty within that constraint. Since the buffer dynamic reflects the mismatch between bitrate and throughput, HCA adjusts bitrate by controlling the buffer occupancy at a low but non-empty level, thus achieving bitrate maximization and rebuffering minimization at the same time.

To accurately control the buffer occupancy in unstable networks, HCA borrows from the design of VoD ABRs. Existing adaptations are mainly based on feedforward which maximizes QoE using throughput predictions [4, 5], or based on feedback which relies on buffer occupancy without prediction [6, 7]. Feedforward can adapt to future network changes but the performance is sensitive to prediction error, whereas the adaptation of feedback is robust against throughput fluctuations but is reactive. Inspired by this, HCA achieves accurate control by using a *hybrid control* which makes rapid reaction to throughput changes by feedforward control and resists the impact of prediction error by feedback control.

In detail, HCA designs a reference buffer dynamic trajectory with the points on which evolving towards the target level. The buffer occupancy is controlled to follow the trajectory to move to the target. By using hybrid control, HCA first computes the bitrate that needed to reach the desired

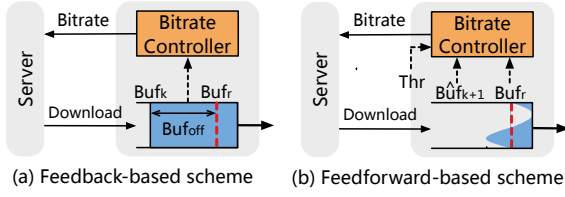


Fig. 1. Motivation

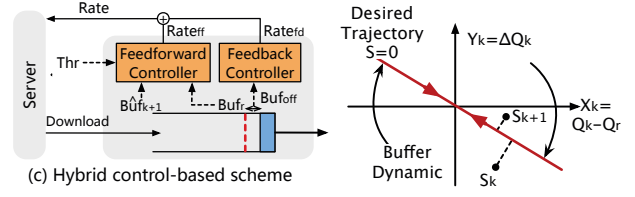


Fig. 2. Basic Idea

trajectory based on predictions of future network condition (throughput and its variation), and then correct the bitrate with considering the current buffer state so that the actual buffer output can be bounded within a limited range of the target.

HCA is evaluated on a reference video player Dash.js with given a small buffer capacity using realistic throughput measurements. Experiment results show that HCA achieves 98% zero rebuffering of all tested networks and the selected bitrates match the average level of existing VoD ABRs.

In the rest of paper, we introduce the background and motivation in Section 2. Section 3 formulates the adaptation problem. Section 4 develops the HCA algorithm. Evaluation experiments are presented in Section 5. Section 6 introduces the related work. The paper is concluded in Section 7.

2. BACKGROUND AND MOTIVATION

Overview of HAS: HAS is currently the most popular technology for streaming VoD and live videos. It can not only stream cost-effectively to a large audience by using CDNs and HTTP servers, but also ensure good QoE by adapting bitrate to varying bandwidth. Live streaming requires low delay but VoD streaming has no requirement. To better support live streaming, HAS provides low delay option by leveraging HTTP chunked transfer, which delivers chunk in unit of smaller segment so that the chunk can be played before it is completely downloaded [1]. The core of HAS is ABR algorithm, which heavily influences the achieved QoE. ABR selects bitrate based on throughput predictions and buffer occupancy with the goal of optimizing QoE, including maximizing bitrate and minimizing rebuffering [6, 7].

State-of-the-art ABR algorithms: Existing ABRs are mainly designed for VoD streaming, which can be roughly divided into feedback-based and feedforward-based algorithms.

As shown in Figure 1(a), feedback-based ABRs [6, 7] adjust bitrate to reduce the offset between the current buffer level and the target. By continuously taking corrective actions, feedback is robust against throughput fluctuations. However, it is also reactive. Without making prediction, the corrective action is taken only after there is a buffer offset. Feedforward-based approach [4, 8, 5] predicts future throughputs and selects the bitrate that matches the throughput or outputs the maximum QoE. The idea is illustrated in Figure 1(b). By adapting to future network condition, feedforward-based

ABRs are able to make rapid reaction to network changes. However, the performance is sensitive to prediction error.

Both of the two methodologies might induce significant buffer occupancy variation in rapidly changing networks. However, they can provide good QoE for VoD streamings because the impact of fluctuations can be absorbed by large buffer occupancy so as to not lead to rebuffering.

Challenges of designing a low delay ABR: Due to the fluctuating network throughput and the conflicting QoE optimization goals, picking a correct bitrate is hard for both VoD streaming and live streaming. However, the stringent low delay requirement poses more challenges for live streaming to provide good QoE. The new challenges are (1) low delay requires small playback buffer occupancy, but which impairs the resistance to throughput fluctuations and increases the risk of rebuffering. (2) increasing video quality will increase the risk of rebuffering and long delay, because minimizing rebuffering and delay would constantly select the lowest bitrate.

Motivation and basic idea: To achieve both good QoE and low delay, ABR algorithm should use a small buffer capacity and under this limitation to select the highest safe bitrate without exhausting the buffer. Since the buffer occupancy increases at the speed of network throughput and decreases at the speed of video bitrate, the selected bitrate fully utilizes the network capacity if the buffer occupancy is constant. Considering that the buffer dynamic not only reflects the bandwidth utilization but also relates to rebuffering, the QoE optimization of low delay streaming can be solved by controlling the buffer occupancy at a low but non-empty level. The key to realizing this potentiality is accurately controlling the buffer occupancy. Motivated by the analysis of existing VoD ABRs, we propose to use the hybrid of feedback control and feedforward control to achieve accurate control. The feedforward controller is used to eliminate the effect of large throughput variation by taking a corrective action ahead of time, and the feedback controller focuses on dealing with the prediction error. By using hybrid control, we can decouple the response to throughput variations and the resistance to disturbances, achieving both robustness and quick response.

3. PROBLEM FORMULATION

Assume there are N versions for video content with different bitrates. All versions of video are partitioned into K

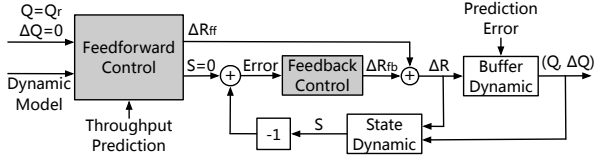


Fig. 3. Overview of HCA

chunks, each of which has the same playback time of V . Let $C_k(R_k)$ be the size of chunk k encoded at bitrate R_k .

Let Q_k denote the buffer occupancy at the start of downloading chunk k and T_k be the average download speed. Each second time, a second video is dequeued from the buffer. When a chunk downloaded, the buffer is added with V seconds. Therefore, the buffer evolves as $Q_{k+1} = \max\{Q_k - C_k(R_k)/T_k, 0\} + V$, where $Q_k \in [V, Q_{max}]$. Let X_k denote the offset of the buffer occupancy to the target level, i.e., $X_k = Q_k - Q_r$, and Y_k denote the buffer occupancy variation in a chunk download interval, i.e., $Y_k = Q_k - Q_{k-1}$. The state-space model uses (X_k, Y_k) to describe the system state. Thus, the ideal state is $(X_k, Y_k) = (0, 0)$, i.e., the origin of the state space. Figure 2 illustrates it.

For convenience, this paper uses $R_k V$ to substitute $C_k(R_k)$, but which should use the actual chunk size in practice. With assuming $T_k = T_{k-1}$ and $\hat{T}_k$ being the estimation, the following equation can be obtained from the buffer model.

$$\Delta Q_{k+1} = \Delta Q_k - \frac{V}{\hat{T}_k} \Delta R_k + \xi_k \quad (1)$$

where ξ_k is the uncertainty term, representing the impact of throughput variation and estimation error on buffer dynamic. With Eq.(1), the system dynamic can be formulated as

$$\begin{aligned} X_{k+1} &= X_k + Y_{k+1} \\ Y_{k+1} &= Y_k - \frac{V}{\hat{T}_k} \Delta R_k + \xi_k \end{aligned} \quad (2)$$

With throughput estimation $\hat{T}_k$ and the uncertainty range $[\xi_{min}, \xi_{max}]$ as input, this formulation outputs the bitrate change ΔR_k that is able to control the buffer occupancy to reach and stay at the target level Q_r .

4. HYBRID CONTROL-BASED ABR

With the formulation in Section 3, bitrate is selected by controlling (X_k, Y_k) at $(0, 0)$. To achieve accurate control, we regulates the evolution of (X_k, Y_k) not just the instantaneous state of (X_k, Y_k) . HCA designs a trajectory that evolves towards the origin and controls the system state to move along it. Figure 3 illustrates the framework of HCA. After designing a desired trajectory $S_k = f(X_k, Y_k)$, there are four steps to determine the bitrate R_k for chunk k :

(1) *Predict Throughput*: Predict the throughput $\hat{T}_k$ and the range of error $\xi_k \in [\xi_{min}, \xi_{max}]$.

(2) *Generate feedforward signal*: Given current state (X_k, Y_k) , compute the signal ΔR_{ff} needed to output the desired behavior $S_{k+1} = f(X_{k+1}, Y_{k+1}) = 0$ based on the system model in Eq.(2) and the throughput prediction $\hat{T}_k$.

(3) *Generate feedback signal*: ΔR_{fd} is set as a function of current state (X_k, Y_k) to decrease the current distance to the desired trajectory in the condition of $\xi_k \in [\xi_{min}, \xi_{max}]$.

(4) *Output bitrate decision*: With the two control signals combined, the requested video bitrate R_k is the highest available bitrate lower than $R_{k-1} + \Delta R_{ff} + \Delta R_{fd}$.

4.1. Throughput Prediction

RBC requires the predictor to make real-time online prediction of throughput and estimation of prediction error variance. Since this paper is not to design a new predictor, we rely on existing solutions. A suitable and robust method is the Kalman Filter predictor proposed in [9].

Let $\theta_k = -V/T_k$ denote the variable to be estimated. As the network condition is fairly stable within seconds, the evolution of θ can be modeled as $\theta_{k+1} = \theta_k + \omega_k$ with ω_k representing the noise. We use $\Delta Y_k = Y_k - Y_{k-1}$ as the measurement of θ_k . Based on the relation between Y_k and T_k shown in Eq.(2), the model of Kalman Filter is

$$\begin{aligned} \theta_{k+1} &= \theta_k + \omega_k \\ \Delta Y_{k+1} &= \Delta R_k \theta_k + \xi_k \end{aligned} \quad (3)$$

Previous studies [8, 10] show that throughput evolution is largely Gaussian. Therefore, ω_k and ξ_k are both modeled as Gaussian process with zero mean. The variances are $\Omega = E[\omega^2(k)]$ and $Z_k = E[\xi^2(k)]$. The estimation of Z is recursively updated by an exponential moving average filter,

$$Z_k = \mu Z_{k-1} + (1 - \mu)[\Delta Y_k - \Delta R_{k-1} \theta_{k-1}]^2 \quad (4)$$

where μ is the weight factor. The estimation of $\hat{\theta}_k$ is given by

$$\begin{aligned} \hat{\theta}_k &= \hat{\theta}_{k-1} + G_k (\Delta Y_k - \Delta R_{k-1} \hat{\theta}_{k-1}) \\ G_k &= \frac{(P_{k-1} + \Omega) \Delta R_k}{\Delta R_k^2 (P_{k-1} + \Omega) + Z_k} \\ P_k &= (1 - G_k \Delta R_k) (P_{k-1} + \Omega) \end{aligned} \quad (5)$$

Finally, the throughput prediction is $\hat{T}_k = -V/\hat{\theta}_k$ with $\hat{\theta}_k$ obtained through Eq.(5). The range of error ξ_k is estimated by $[-Z_k, Z_k]$ where Z_k is estimated according to Eq.(4).

4.2. Design of Bitrate Controller

The first step is designing the reference evolution trajectory. Let $S_k = X_k + \lambda Y_k$ with $\lambda < 1$. Then, the desired trajectory is defined as $S_k = 0$. A negative S_k means that the current state (X_k, Y_k) is below the trajectory while $S_k > 0$ suggests that (X_k, Y_k) lies above the line. It proves that all points at the reference trajectory converge to the origin at the end of this section. Therefore, as long as the bitrate controller

constrains the system state to stay at the desired trajectory, i.e., adjusting bitrate to achieve $S_{k+1} = 0$, the buffer state will follow the line sliding to the origin. This goal is achieved by combining feedforward and feedback signals.

According to Eq.(2), the evolution of S_{k+1} is

$$S_{k+1} = \lambda S_k + Y_k - \frac{(\lambda + 1)V}{\hat{T}} \Delta R_k + (\lambda + 1)\xi_k \quad (6)$$

By setting $S_{k+1} = 0$, we can obtain the feedforward signal, i.e., $\Delta R_{ff} = [(\lambda S_k + Y_k)\hat{T}]/[(\lambda + 1)V]$. The feedback controller takes a proportional form $\Delta R_{fd} = \alpha X_k$. If $X_k S_k > 0$, $\alpha = \alpha_1$. If $X_k S_k < 0$, $\alpha = \alpha_2$. By combining ΔR_{ff} and ΔR_{fd} , the bitrate decision ΔR_k is

$$\Delta R_k = \frac{(\lambda S_k + Y_k)\hat{T}}{(\lambda + 1)V} + \alpha X_k \quad (7)$$

The parameter α should be set to ensure that the system state always converges to the reference trajectory. We determine the value of α_1 and α_2 by solving $|S_{k+1}| < |S_k|$, namely the next distance to the reference line is smaller than the present distance. The idea is shown in Figure 2. To solve it, we substitute Eq.(7) into Eq.(6) and obtain $S_{k+1} = -[(\lambda + 1)V/\hat{T}]\alpha X_k + (\lambda + 1)\xi_k$. With it, the range of α_1 and α_2 that satisfies $|S_{k+1}| < |S_k|$ can be obtained. Considering ξ_k is an uncertain variable within a bounded range, the range of values for α should be further narrowed down so that $|S_{k+1}| < |S_k|$ holds for any possible value of ξ_k in the estimated range $[\hat{\xi}_{min}, \hat{\xi}_{max}]$. It follows that

$$\begin{aligned} \frac{\hat{T}\hat{\xi}_{max}}{VX_k} - \frac{\hat{T}S_k}{V(\lambda + 1)X_k} < \alpha_1 < \frac{\hat{T}\hat{\xi}_{min}}{VX_k} + \frac{\hat{T}S_k}{V(\lambda + 1)X_k} \\ \frac{\hat{T}\hat{\xi}_{max}}{VX_k} + \frac{\hat{T}S_k}{V(\lambda + 1)X_k} < \alpha_2 < \frac{\hat{T}\hat{\xi}_{min}}{VX_k} - \frac{\hat{T}S_k}{V(\lambda + 1)X_k} \end{aligned} \quad (8)$$

Proof of convergence: Let's discuss the evolution of X_{k+1} when current state (X_{k+1}, Y_{k+1}) is on the reference trajectory. Given $S_k = 0$, it satisfies $Y_k = -\lambda X_k$. Based on Eq.(2) and the result of ΔR_{ff} , we have $X_{k+1} = (1 - \lambda + \lambda X_k/(\lambda + 1))$. It is obvious that $|X_{k+1}| < |X_k|$ because $1 - \lambda + \lambda/(\lambda + 1) < 1$. With the Lyapunov function defined as $L_k = X_k^2/2$, it obtains that $\Delta L_{k+1} = \frac{1}{2}X_{k+1}^2 - \frac{1}{2}X_k^2 < 0$, ($X_k \neq 0$), which proves that system states located on $S = 0$ asymptotic converges to $X = 0$. Namely, the buffer occupancy converges to the target level.

5. EVALUATION

To evaluate HCA, we carry out experiments on a reference DASH player using realistic network traces.

5.1. Experimental Setup and Adaptation Methods

Throughput traces: We use an HSDPA mobile dataset [11] and a proprietary dataset from KwaiShou [12], which is

a leading commercial video provider in China. The HSDPA dataset comprises measurements of video streaming throughput of a moving device in Telenor's 3G/HSDPA mobile wireless network in Norway. For evaluation experiments, we generate 300 traces with 300 continuous 1s throughput measurements from randomly selected traces for each dataset.

Implementation and setup: We use Nginx as the basis of server deployment and Dash.js as the video player [13]. To implement algorithms, the original Dash.js is modified using the methods in [4]. Our evaluation uses video "Envivio" from [14] which consists of 100 2s chunks with each encoded in {0.35, 0.6, 1, 2, 3} Mbps. The buffer capacity is $Q_{max} = 5s$.

ABR algorithms: HCA selects bitrate by Eq.(7) with $Q_r = 2s$, $\lambda = 0.75$ and α bounded by Eq.(8). Throughput predictor adopts the setting of $\Omega = 10^5$ and $\mu = 0.7$. For comparison, several typical VoD algorithms are evaluated:

Feedback-based approaches:

(1) BBA [6]: selects bitrate by a function of buffer occupancy with a reservoir of 2s and a cushion of 3s. The goal is to control the buffer occupancy between 2s and 5s.

(2) BOLA [7]: is designed based on Lyapunov optimization, the bitrate decision of which is essentially a function of current buffer occupancy.

Feedforward-based approaches:

(3) MPC [4]: maximizes the designed QoE utility over 5 chunks with throughput predictions.

(4) RobustMPC (denoted as RMPC) [4]: uses the same approach as MPC but mitigates the effect of prediction error by normalizing throughput predictions with the maximum prediction error observed in the past 5 chunks.

(5) RB: picks the highest bitrate less than the estimated throughput of the next chunk.

QoE metrics: (a) average bitrate; (b) total rebuffering time; (c) QoE utility, computed by $QoE = Rate - \gamma \times RebufTime$. Since rebuffering not only degrades QoE but also increases the delay, so we adopt the "Avoid Rebuffering" setting, i.e., $\gamma = 6$, as recommended in [4].

5.2. Results

Statistical Performance of QoE and Buffer Dynamic:

Figure 4 presents the CDF of per-session average bitrate, total rebuffering time. It observes that HCA exhibits great advantages in reducing rebuffering, with zero rebuffering session ratio being 96% in HSDPA and 100% in KwaiShou network. The video quality also rivals to existing algorithms. The most competitive algorithm is BOLA and RMPC. BOLA achieves good performance in reducing rebuffering but the bitrate selection is too conservative. RMPC provides a slightly higher video bitrate but the rebuffering is much severer than HCA.

We further zoom in on the buffer dynamics to better understand the performance differences. Since small buffer capacity limits the buffer dynamic which thus can not reflect the control performance of algorithms, we use a large buffer

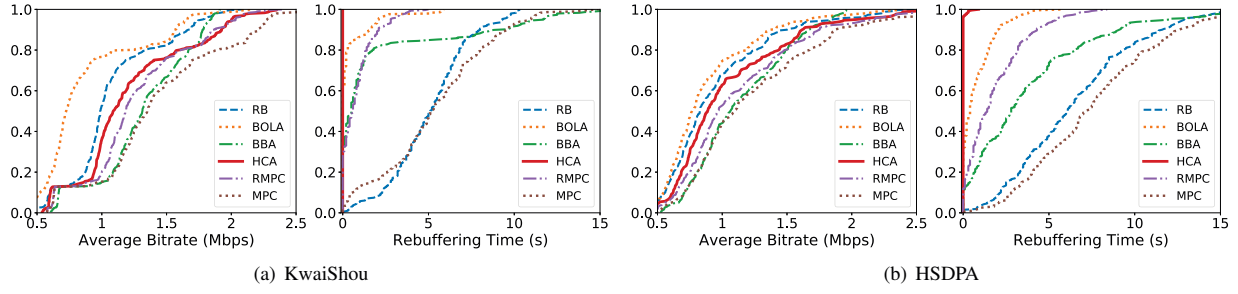


Fig. 4. Statistical performance of QoE metrics

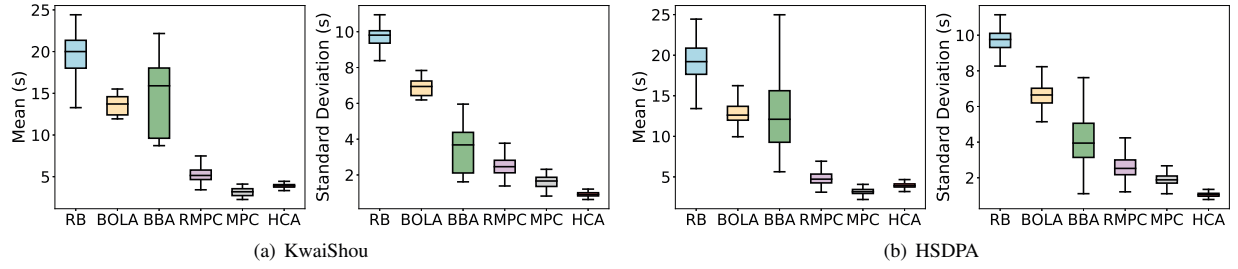


Fig. 5. Statistical performance of buffer occupancy

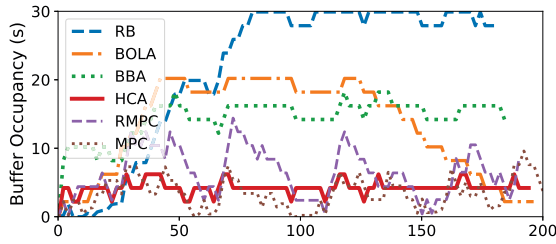


Fig. 6. Detailed buffer dynamics of ABRs in a session

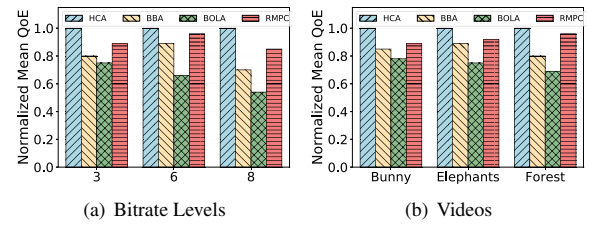


Fig. 7. Sensitivity of HCA to video characteristics

capacity of 30s to re-run ABR algorithms. Figure 5 presents the statistical results of buffer dynamic, and Figure 6 uses one session to illustrate the detail. As shown, HCA is able to accurately control the buffer occupancy at a low but non-empty level. It controls the buffer occupancy at around 4s and leads to the minimum standard deviation with about 1s on average. Although the buffer occupancies of RMPC and MPC are also small, their variation is not trivial, thus inducing frequent rebuffering. Since BOLA and BBA both fully rely on buffer occupancy to adapt bitrate, they all keep large buffering to absorb the impact of throughput fluctuations. When the buffer capacity is limited to a small size, feedback controllers can result in either long rebuffering or conservative bitrate.

Sensitivity to video properties: We further evaluate the sensitivity of HCA to available bitrate set and video type. Due to the limited space, Figure 7 only presents the *QoE* utility of several competitive algorithms, i.e., BBA, BOLA and RMPC. Results are normalized against the performance of

HCA. First, ABR algorithms are performed on three different bitrate sets, including $\{0.5, 1.5, 3\}$ Mbps with 3 levels, $\{0.3, 0.75, 1.2, 1.85, 2.85, 4.3\}$ Mbps with 6 levels and $\{0.35, 0.6, 1, 1.5, 2, 2.5, 3, 3.5\}$ Mbps with 8 levels. Figure 7(a) shows that HCA provides the best QoE. Since video property influences the adaptation process, we also test the performance of HCA on three new videos from dataset [15]. Figure 7(b) reveals that HCA keeps its advantage for all tested videos.

Impact of algorithm parameters: Figure 8 analyzes the performance of HCA with regard to key parameters – weight factor λ and target buffer occupancy Q_r . It shows that QoE reaches the maximum when $\lambda = 0.75$ and $Q_r = 2s$.

6. RELATED WORK

There have been several ABR research for low delay HTTP streaming. Le et al. [16] propose an algorithm for live streaming by optimizing JND (Just Noticeable Difference).

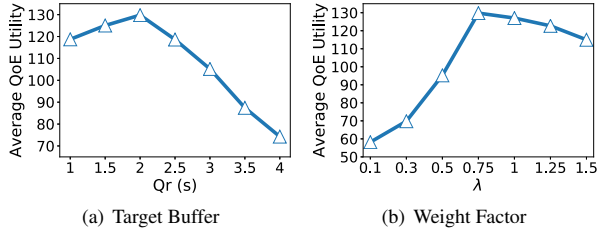


Fig. 8. Impact of algorithm parameters on HCA

However, the algorithm outputs a non-trivial buffer occupancy which does not satisfy low-delay requirement. [17] designs LOLYPOP to improve the throughput prediction, which is still sensitive to prediction error.

7. CONCLUSION

In this work, we present a low-delay bitrate adaptation solution HCA, which optimizes QoE with the limitation of small buffer by controlling the buffer occupancy at a small but non-empty level. To accurately control the buffer occupancy in unstable network, HCA uses the hybrid of feedback control and feedforward control, achieving both robustness and rapid reaction. Over a large throughput dataset, evaluation experiments demonstrate that HCA achieves zero rebuffering for 98% of all traces while providing high bitrate.

8. ACKNOWLEDGMENTS

The authors gratefully acknowledge the anonymous reviewers for their constructive comments. This work is supported in part by the National Key Research and Development Program of China (No.2018YFB1700103), and by National Natural Science Foundation of China (NSFC) under Grant 61872208.

9. REFERENCES

- [1] *Low Latency HLS*, <https://www.theoplayer.com/blog/low-latency-hls-lhls>.
- [2] Iraj Sodagar, “The mpeg-dash standard for multimedia streaming over the internet,” *IEEE MultiMedia*, 2011.
- [3] *Why YouTube and Netflix use MPEG-DASH in HTML5*, <https://bitmovin.com/status-mpeg-dash-today-youtube-netflix-use-html5-beyond/>.
- [4] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli, “A control-theoretic approach for dynamic adaptive video streaming over http,” in *Proc. ACM SIGCOMM*, 2015.
- [5] Junchen Jiang, Vyas Sekar, and Hui Zhang, “Improving fairness, efficiency, and stability in http-based adaptive video streaming with festive,” in *Proc. ACM CoNEXT*, 2012.
- [6] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson, “A buffer-based approach to rate adaptation: Evidence from a large video streaming service,” in *Proc. ACM SIGCOMM*, 2014.
- [7] Kevin Spiteri, Rahul Ugaonkar, and Ramesh K Sitaraman, “Bola: near-optimal bitrate adaptation for online videos,” in *Proc. IEEE INFOCOM*, 2016.
- [8] Yi Sun, Xiaoqi Yin, Junchen Jiang, Vyas Sekar, Fuyuan Lin, Nanshu Wang, Tao Liu, and Bruno Sinopoli, “Cs2p: Improving video bitrate selection and adaptation with data-driven throughput prediction,” in *Proc. ACM SIGCOMM*, 2016.
- [9] Bo Wang and Fengyuan Ren, “Towards forward-looking online bitrate adaptation for dash,” in *Proceedings of the 2017 ACM on Multimedia Conference*. ACM, 2017.
- [10] Zahaib Akhtar, YunSeong Nam, Ramesh Govindan, Sanjay Rao, Jessica Chen, Ethan Katz-Bassett, Bruno Ribeiro, Jibin Zhan, and Hui Zhang, “Oboe: auto-tuning video abr algorithms to network conditions,” in *Proc. ACM SIGCOMM*, 2018.
- [11] *HSDPA dataset*, <http://home.ifi.uio.no/paalh/dataset/hsdpa-tcp-logs>.
- [12] *KwaiShou*, <https://www.kuaishou.com/>.
- [13] *Dash-Industry-Forum*, *dash.js*, <https://github.com/Dash-Industry-Forum/dash.js/wiki>.
- [14] *DASH JavaScript reference client landing page 1.4.0.*, <http://dashif.org/reference/players/javascript/1.4.0/samples/dash-if-reference-player/index.html>.
- [15] Stefan Lederer, Christopher Müller, and Christian Timmerer, “Dynamic adaptive streaming over http dataset,” in *Proc. ACM MMSys*, 2012.
- [16] H. T. Le, H. N. Nguyen, N. Pham Ngoc, A. T. Pham, H. Le Minh, and T. C. Thang, “Quality-driven bitrate adaptation method for http live-streaming,” in *IEEE ICCW*, 2015.
- [17] Konstantin Miller, Abdel-Karim Al-Tamimi, and Adam Wolisz, “Qoe-based low-delay live streaming using throughput predictions,” *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 13, no. 1, pp. 4:1–4:24, 2016.